



**Koneru Lakshmaiah Education Foundation**

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, [www.klh.edu.in](http://www.klh.edu.in)

## **DATABASE SYSTEMS ENGINEERING AND DISTRIBUTED BACKEND DEVELOPMENT (25CS1302E)**

*Y25 · 2026–2027 · Trimester 4 · Section 13 · Team 03*

# **CAR RENTAL & MOBILITY PLATFORM**

## **Review-1**

### **Team Members**

**2520080037 – R. Rohit**

**2520090027 – G. Bhupathi**

**2520090082 – S. Anjani Shreyaj**

# Contents of Review-1



01  
Abstract



02  
Problem Statement



03  
Objectives



04  
Societal Need /  
Motivation



05  
Feasibility Analysis



06  
Novelty



07  
CO Mapping



08  
Technologies Used



09  
System Flow

# Abstract



## Project Overview

The Car Rental & Mobility Platform is a microservice-based backend that connects customers, vehicles, bookings, and payments on one platform.

It uses PostgreSQL for transactional data, MongoDB for the flexible vehicle catalog and reviews, and pgvector for similarity search.

It gives real-time availability and tracking, secure role-based access, and smart vehicle recommendations.

## What the System Manages



**Bookings & Payments**



**Vehicle Catalog**



**Real-Time Tracking**



**Recommendations**



**Access & Monitoring**

# Problem Statement



Existing rental systems use one database for everything, so they struggle with consistency, scaling, live tracking, and smart recommendations.



## Single-Database Limits

One database for bookings, listings, and reviews does not scale well.



## No Real-Time Visibility

Availability and vehicle location are not updated live.



## Weak Recommendations

Only keyword filters are used; similar cars are not suggested.

# Objectives

1

To combine PostgreSQL, MongoDB, and pgvector in one data layer



2

To build a FastAPI gateway with Node.js and Spring Boot services



3

To keep booking, payment, and inventory consistent using the Saga pattern



4

To recommend similar vehicles using pgvector similarity search



5

To deploy with Docker and Kubernetes, with monitoring and CI/CD



# Societal Need / Motivation



**Makes vehicle  
rental simple  
and accessible**



**Gives live  
availability and  
vehicle  
tracking**



**Ensures safe,  
reliable  
bookings and  
payments**



**Supports  
affordable,  
pay-per-use  
mobility**



**Promotes  
shared  
mobility and  
fewer private  
cars**

# Feasibility Analysis



## Technical Feasibility

Uses proven tools: FastAPI, Node.js, Spring Boot, PostgreSQL, MongoDB, and pgvector.



## Economic Feasibility

All tools are open-source and run in Docker, so there is no licensing cost.



## Operational Feasibility

Customers, agents, and admins get simple role-based access through one gateway.



## Academic Suitability

Covers SQL, NoSQL, vector search, microservices, and Saga, which suits this course.

# Novelty of the Project



## Polyglot Car Rental Platform

The right database for each workload, in one service-oriented backend



Polyglot persistence: PostgreSQL, MongoDB, pgvector



Similarity-based vehicle recommendations



Saga-based consistency across booking and payment



Live tracking with Socket.io and Redis

# CO Mapping

| CO  | Mapping in the Project                                                                          |
|-----|-------------------------------------------------------------------------------------------------|
| CO1 | PostgreSQL schema for bookings and payments, SQL queries, joins, and reports                    |
| CO2 | SQL vs NoSQL: PostgreSQL for transactions, MongoDB for catalog and reviews, pgvector for search |
| CO3 | FastAPI gateway with REST APIs, JWT/OAuth2, validation, and Swagger / OpenAPI docs              |
| CO4 | Backend services: Node.js tracking, Spring Boot inventory and pricing, auth, and admin modules  |
| CO5 | Microservice architecture using the Saga pattern and event-driven consistency                   |
| CO6 | Docker, Kubernetes, GitHub Actions CI/CD, and Prometheus / Grafana monitoring                   |

# Technologies Used and System Flow

## Technologies Used



**Backend**  
FastAPI, Node.js, Spring Boot



**Databases**  
PostgreSQL, MongoDB, pgvector



**Real-Time & Auth**  
Socket.io, Redis, JWT / OAuth2



**Deployment**  
Docker, Kubernetes, CI/CD



**Monitoring**  
OpenTelemetry, Prometheus, Grafana

## System Flow

1

User logs in through the FastAPI gateway

2

Gateway routes the booking request to services

3

Inventory service checks live availability

4

Saga coordinates booking, payment, and inventory

5

**pgvector recommends similar vehicles**

6

Live tracking via Socket.io; metrics in Grafana